

Python Script

Eloïse Azzola

August 15, 2023

1 Merge Refinitiv & Trucost database

1.1 Import libraries

```
[3]: import pandas as pd
import numpy as np
from unicode import unicode
import json
```

1.2 Functions

```
[4]: # Function to rename a string named 'FY0' to 2022, 'FY-1' to 2021, 'FY-2' to 2020, etc.
def rename_fiscal_year(fiscal_year):
    current_year = 2022
    if fiscal_year == 'FY0':
        return current_year
    elif fiscal_year.startswith('FY-'):
        year_offset = int(fiscal_year[3:])
        renamed_year = current_year - year_offset
        return renamed_year
    else:
        return fiscal_year
```

1.3 Constants

```
[5]: # Export results in Excel files (can take time)
EXPORT = False

# Dictionary of words to erase in the companies name. Use Regex syntax
comp_name_dict_erase = {
    ',': '',
    '\\.': '',
    '-': '',
    '"': '',
    '\\bthe\\b': '',
    '\\bsjsc\\b': ''
}
```

```
'\\bcorporation\\b': '',
'\\bcorp\\b': '',
'\\bberhard\\b': '',
'\\bberhad\\b': '',
'\\bbhd\\b': '',
'\\blimited\\b': '',
'\\bltd\\b': '',
'\\bplc\\b': '',
'\\bco\\b': '',
'\\bcompany\\b': '',
'\\bpt\\b': '',
'\\btbk\\b': ''}
```

1.4 Refinitiv controls

1.4.1 Import data

```
[6]: # Importing data from Refinitiv.
# Has 2 levels of column names and 6 index to describe a company.
refinitiv_controls = pd.read_excel('data/GridExport_August_15_2023_12_16_28.
→xlsx', header=[0,1], index_col=[0,1,2,3,4,5])
# Rename the index because they disappeared.
refinitiv_controls.index.names=['Identifier (RIC)', 'Company Name', 'Ticker_
→Symbol', 'Date Became Public', 'NAICS Industry Group Name', 'Business_
→Description']
refinitiv_controls.columns.names=['Variable', 'Fiscal Year']
```

1.4.2 Clean data

```
[7]: # Remove SD column
# refinitiv_controls.drop('Earnings Per Share - Standard Deviation\n(USD)\nIn_
→the last 18 Y', axis=1, inplace=True, level=0)
# Rename the column names based on fiscal years
refinitiv_controls.rename(columns=rename_fiscal_year, inplace=True, level=1)
# Remove year 2022
# refinitiv_controls.drop('2022', axis=1, inplace=True, level=1)
```

1.4.3 Clean compagny names

```
[8]: # Reset index to column to be able to do modification to it
refinitiv_controls.reset_index(level="Company Name", inplace=True)

# Clean company names of the trucost dataset. Transform to ASCII, lower and_
→erease characters
company_name_clean = refinitiv_controls['Company Name'].apply(unidecode).str.
→lower().replace(comp_name_dict_erase, regex=True)
```

```

# Insert new compagny name in new column
refinitiv_controls.insert(1, 'Company Name Clean', company_name_clean)

# Set two column as index
refinitiv_controls.set_index(keys=['Company Name','Company Name Clean'],
    ↳append=True, inplace=True)
# Put Company Name as first index
refinitiv_controls = refinitiv_controls.reorder_levels([5,6,0,1,2,3,4])
# Remove column transformed into index from columns list
refinitiv_controls.columns = refinitiv_controls.columns.remove_unused_levels()

```

1.4.4 Add year 2004 to 2022 if not present

```

[9]: for col_name in refinitiv_controls.columns.levels[0]:
      for year in range(2004, 2023):
          if year not in refinitiv_controls[col_name].columns:
              refinitiv_controls[col_name, year] = np.nan

# Reorder columns by name because new columns are put at the end
refinitiv_controls = refinitiv_controls.sort_index(axis=1, level=0)

```

1.4.5 Reshape data

```

[10]: refinitiv_controls_stacked = refinitiv_controls.stack(level=1, dropna=False)

```

1.5 Refinitiv returns

1.5.1 Import data

It's necessary to add year/month in the column header for returns per month

Total Return by Month (01.01.2004-31.12.2022)												
2004												
1	2	3	4	5	6	7	8	9	10	11	12	1
-6.9%	3.5%	-6.5%	4.3%	-5.5%	-4.6%	1.5%	7.4%	7.5%	-5.0%	3.7%	-1.8%	-1.4%

```

[11]: refinitiv_returns = pd.read_excel('data/Refinitiv_Returns.xlsx',
    ↳sheet_name="Agriculture Comp, Full + Ticker", header=[0,1,2],
    ↳index_col=[0,1,2,3,4,5])
# Rename index and columns
refinitiv_returns.index.names=['Identifier (RIC)', 'Company Name', 'Date Became_
    ↳Public', 'NAICS Industry Group Name', 'Business Description', 'Ticker Symbol']
refinitiv_returns.columns.names=['Variable', 'Fiscal Year', 'Fiscal Month']

```

1.5.2 Clean data

```

[12]: # Remove return per year columns
refinitiv_returns_month = refinitiv_returns.drop('Total Return by Year (01.01.
    ↳2004-31.12.2022)',axis=1,level=0)
# Remove year 2004, 2022

```

```
# refinitiv_returns_month.drop([2004,2022],axis=1,inplace=True, level=1)
```

1.5.3 Generate yearly returns statistics

```
[13]: # Gerenate column with available fiscal months per year
refinitiv_returns_count = refinitiv_returns_month.groupby(axis=1, level=1).
    ↪count()
# Add header name
refinitiv_returns_count = pd.concat([refinitiv_returns_count], axis=1,
    ↪keys=["Available fiscal months"])
```

```
[14]: # Generate mean for each year
refinitiv_returns_mean = refinitiv_returns_month.groupby(axis=1, level=1).mean()
# Add header name
refinitiv_returns_mean = pd.concat([refinitiv_returns_mean], axis=1,
    ↪keys=["Return Mean"])
```

```
[15]: # Generate composed return for each year
refinitiv_returns_composed = (refinitiv_returns_month+1).groupby(axis=1,
    ↪level=1).prod()-1
# Add header name
refinitiv_returns_composed = pd.concat([refinitiv_returns_composed], axis=1,
    ↪keys=["Composed Return"])
```

```
[16]: # Generate Standard Deviation for each year
refinitiv_returns_std = refinitiv_returns_month.groupby(axis=1, level=1).std()
# Add header name
refinitiv_returns_std = pd.concat([refinitiv_returns_std], axis=1, keys=["Return_
    ↪Standard Deviation"])
```

```
[17]: # Put every result in the same dataframe
refinitiv_returns_year = pd.concat([refinitiv_returns_count,
    ↪refinitiv_returns_mean, refinitiv_returns_composed, refinitiv_returns_std],
    ↪axis=1)
```

1.5.4 Reshape data

```
[18]: refinitiv_returns_year_stacked = refinitiv_returns_year.stack(level=1,
    ↪dropna=False)
```

1.5.5 Create lagged composed return

```
[19]: # Create lagged year
refinitiv_returns_year_stacked['Lagged Composed Return'] =
    ↪refinitiv_returns_year_stacked.groupby('Company Name')['Composed Return'].
    ↪shift(-1)
```

1.6 Merge Controls & Returns Refinitiv datas

```
[20]: refinitiv_data = refinitiv_controls_stacked.reset_index().  
      ↪join(refinitiv_returns_year_stacked.reset_index(drop=True)).  
      ↪set_index(refinitiv_controls_stacked.index.names)
```

1.7 Export Refinitiv datas

```
[21]: if EXPORT:  
      refinitiv_data.to_excel('output/Refinitiv_merged.xlsx')
```

1.8 Trucost

1.8.1 Import data

```
[22]: trucost_data = pd.read_excel('data/Trucost 2004-2021 no duplicates.xlsx',  
      ↪header=[0], index_col=[0,1,2,3,4,5,6,7,8])
```

1.8.2 Clean data

1.8.3 Clean compagny names

```
[53]: # Reset index to column to be able to do modification to it  
      trucost_data.reset_index(level="Company Name", inplace=True)  
  
      # Use compagny names lookup table to replace company names from Trucost dataset  
      # Dictionary must be in the form of {'trucost_name': 'refinitiv_name'}  
      with open('data/name_lookup.json', 'r') as fd:  
          comp_name_lookup = json.load(fd)  
      trucost_data['Company Name'] = trucost_data['Company Name'].  
          ↪replace(to_replace=comp_name_lookup)  
  
[54]: # Clean company names of the trucost dataset. Transform to ASCII, lower and  
      ↪erase characters  
      company_name_clean = trucost_data['Company Name'].apply(unidecode).str.lower().  
          ↪replace(comp_name_dict_erase, regex=True)  
  
      # Insert new compagny name in new column  
      trucost_data.insert(1, 'Company Name Clean', company_name_clean)  
  
      # Set two column about Company Name as index  
      trucost_data.set_index(keys=['Company Name', 'Company Name Clean'], append=True,  
          ↪inplace=True)  
      # Put Company Name as first index  
      trucost_data = trucost_data.reorder_levels([8,9,0,1,2,3,4,5,6,7])
```

1.9 Export Trucost data

```
[55]: if EXPORT:
      trucost_data.to_excel('output/Trucost_merged.xlsx')
```

1.10 Merge database

```
[56]: # Merge both database on 'Company Name Clean' and 'Fiscal Year'. Keep only row
      ↳ where data from both database are found
merged_data = trucost_data.reset_index().merge(refinitiv_data.reset_index(),
      ↳ on=['Company Name Clean', 'Fiscal Year'], how='inner',
      ↳ suffixes=['_trucost', '_refinitiv'])
```

```
[57]: # Set the index in the desired order
merged_data = merged_data.set_index(keys=[
    'Company Name Clean',
    'Company Name_trucost',
    'Institution ID',
    'Ticker',
    'Status',
    'Company Type',
    'Country',
    'Year Founded',
    'Web Page',
    'Company Name_refinitiv',
    'Identifier (RIC)',
    'Date Became Public',
    'NAICS Industry Group Name',
    'Business Description',
    'Ticker Symbol',
    'Fiscal Year'])
```

1.11 Export merged database

```
[58]: if EXPORT:
      merged_data.reset_index().to_excel('output/merged.xlsx', index=False)
```

1.12 Export list of company found/not found in refinitiv database

```
[59]: companies_found=merged_data.index.unique(level='Company Name Clean')
      if EXPORT:
          companies_found.to_series().to_excel('output/compagnies_found.xlsx')
```

```
[60]: companies_not_found=trucost_data.drop(labels=companies_found, level='Company
      ↳ Name Clean').index.unique(level='Company Name Clean')
      if EXPORT:
          companies_not_found.to_series().to_excel('output/compagnies_not_found.xlsx')
```

1.13 Print stats on found companies

```
[61]: refinitiv_companies_count = len(refinitiv_data.index.unique('Company Name_
      ↪Clean'))
      trucost_companies_count = len(trucost_data.index.unique('Company Name Clean'))
      print(f'Total companies Refinitiv: {refinitiv_companies_count}\nTotal companies_
      ↪Trucost: {trucost_companies_count}\nMatching companies found:
      ↪{len(companies_found)}')
```

Total companies Refinitiv: 1013

Total companies Trucost: 376

Matching companies found: 163